

Lesson 1A

Parallelism=1

In Apache Flink, even when functions are written as a single chained method call in code (e.g., `dataStream.windowAll(...).reduce(...).print()`), Flink breaks the pipeline into separate operator subtasks (**boxes** in the Web UI graph) at specific stream execution boundaries.

Here is why your `windowAll -> reduce -> print` is separated from the `Source -> Map` box:

1. The Key/Partitioning Boundary (`windowAll`)

- **Source -> Map** operates at a parallelism of 1 (or can scale across multiple parallel instances independently).
- **`windowAll` is a global operation.** By design, `windowAll()` forces all records from across the entire cluster to route into a **single, unified task** to calculate a window over all data.
- To move records from the map task to the `windowAll` task, Flink introduces a partition/hash network edge (`HASH` or `REBALANCE` edge shown on the arrow). Flink **cannot fuse (chain) operators across a network boundary** or partition edge.

2. Why `reduce` and `print` ARE Fused Together inside the Second Box

Looking at the label of your second box:

`TumblingProcessingTimeWindows -> Sink: Print to Std. Out`

Flink **did** fuse the `reduce` function and the `print()` sink together!

- The `reduce()` function executes incrementally *inside* the `TumblingProcessingTimeWindows` trigger.
- Once the 15-second timer fires, the result of the reduction is immediately passed directly in-memory to the `Print` sink.

- Because `reduce` and `print` run in the same task thread without requiring a data re-partitioning, Flink chains them into that single operator box.

Summary

- **Box 1 (Source -> Map):** Generates and transforms data stream records.
- **Network Shuffle (HASH):** Routes data to the single global window operator.
- **Box 2 (WindowAll + Reduce -> Print):** Collects the windowed data, runs the reduce function, and prints the result in a single fused task thread.

Parallelism=2

In this execution graph, Flink split the pipeline into **3 separate operator boxes** instead of 2. Here is why each split happens:

1. Source: string-generator -> Map (Parallelism: 2)

- **Chaining:** The source and `.map(word -> 1L)` both run at **Parallelism = 2**. Since no partitioning change happens between the source and map, Flink fuses them together into a single task.
- **First Split (HASH / Network Boundary):** Because `.windowAll(...)` must evaluate all incoming streams in one central place, data must transition from **Parallelism 2 down to Parallelism 1**. Flink cannot fuse operators across a parallelism change, so it draws a boundary here and routes records via a `HASH` partitioner.

2. TumblingProcessingTimeWindows (Parallelism: 1)

- **Why it's isolated:** `windowAll` forces this operator to run with **Parallelism = 1** to aggregate all elements from both source subtasks.
- **Second Split (REBALANCE / Network Boundary):** After the 15-second window fires and `.reduce()` computes the sum, the result needs to be sent to `.print()`.
- **Why `.print()` wasn't fused here:** The default environment parallelism is set to **2** (`env.setParallelism(2)`). Therefore, `.print()` inherits a **Parallelism of 2**. Since moving data from a task with **Parallelism 1** to a sink with **Parallelism 2** requires re-distributing records across instances, Flink inserts a `REBALANCE` edge and creates a separate vertex for the sink.

3. Sink: mapper-window-word-counter (Parallelism: 2)

- **Custom Name:** `.name("mapper-window-word-counter")` was applied at the end of the chain, which names the sink vertex.
- **Execution:** Every 15 seconds, when the window produces a reduced single record result, the `REBALANCE` channel forwards it to one of the 2 parallel print subtasks to be printed.

Summary Comparison

Aspect	Previous Graph (Default)	Current Graph
Env Parallelism	1	2
Window Parallelism	1 (<code>windowAll</code>)	1 (<code>windowAll</code>)
Sink Parallelism	1 (fused into window)	2 (separated via <code>REBALANCE</code>)
Total Operator Vertices	2 boxes	3 boxes